

Введение в Linux

Отчёт по третьему этапу прохождения внешнего курса

Кулаженкова Яна Сергеевна

2026-05-16

Информация

Вводная часть

Выполнение заданий курса

Результаты

Заключение

1. Информация

- Кулаженкова Яна Сергеевна

- Кулаженкова Яна Сергеевна
- Студентка группы НКАбд-03-25

- Кулаженкова Яна Сергеевна
- Студентка группы НКАбд-03-25
- Российский университет дружбы народов им. Патриса Лумумбы

- Кулаженкова Яна Сергеевна
- Студентка группы НКАбд-03-25
- Российский университет дружбы народов им. Патриса Лумумбы
- 1032253497@rudn.ru

2. Вводная часть

- Редактор vim и bash-скрипты — основа автоматизации в Linux

- Редактор vim и bash-скрипты — основа автоматизации в Linux
- Продвинутый поиск (find, grep, sed) ускоряет работу с файлами

- Редактор vim и bash-скрипты — основа автоматизации в Linux
- Продвинутый поиск (find, grep, sed) ускоряет работу с файлами
- Gnuplot позволяет визуализировать данные

- Редактор vim и bash-скрипты — основа автоматизации в Linux
- Продвинутый поиск (find, grep, sed) ускоряет работу с файлами
- Gnuplot позволяет визуализировать данные
- Управление правами и дисковым пространством — важные навыки администрирования

- **Объект:** продвинутые инструменты командной строки Linux

- **Объект:** продвинутые инструменты командной строки Linux
- **Предмет:** навыки эффективной работы с:

- **Объект:** продвинутые инструменты командной строки Linux
- **Предмет:** навыки эффективной работы с:
 - ▶ текстовым редактором vim

- **Объект:** продвинутые инструменты командной строки Linux
- **Предмет:** навыки эффективной работы с:
 - ▶ текстовым редактором vim
 - ▶ bash-скриптами (переменные, циклы, ветвления)

- **Объект:** продвинутые инструменты командной строки Linux
- **Предмет:** навыки эффективной работы с:
 - ▶ текстовым редактором vim
 - ▶ bash-скриптами (переменные, циклы, ветвления)
 - ▶ поиском (find, grep, sed)

- **Объект:** продвинутые инструменты командной строки Linux
- **Предмет:** навыки эффективной работы с:
 - ▶ текстовым редактором vim
 - ▶ bash-скриптами (переменные, циклы, ветвления)
 - ▶ поиском (find, grep, sed)
 - ▶ визуализацией (gnuplot)

- **Объект:** продвинутые инструменты командной строки Linux
- **Предмет:** навыки эффективной работы с:
 - ▶ текстовым редактором vim
 - ▶ bash-скриптами (переменные, циклы, ветвления)
 - ▶ поиском (find, grep, sed)
 - ▶ визуализацией (gnuplot)
 - ▶ системными утилитами (chmod, wc, du)

Цель: освоение продвинутых навыков работы в Linux.

Задачи: 1. Изучить основные команды редактора vim 2. Научиться писать bash-скрипты (переменные, циклы, условия) 3. Освоить продвинутый поиск файлов и текста 4. Приобрести навыки построения графиков в gnuplot 5. Изучить системные утилиты для управления файлами

3. Выполнение заданий курса

3.1 Текстовый редактор vim — выход

3.1 Текстовый редактор vim
Шаг 5 из 12

?

?

?

?

?

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один элемент

" : ", затем "q", затем "Enter" ✓

"q"

"q", затем "Enter"

"Esc"

" : ", затем "q"

✓ Правильно, молодец!

↶

Дальше

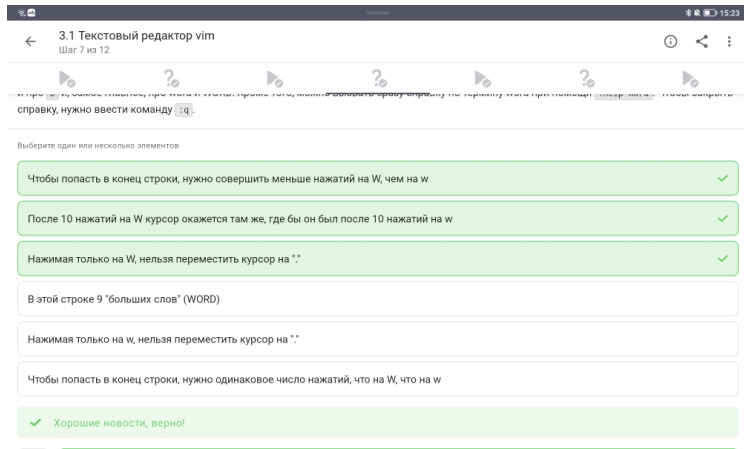
Вопрос: Как выйти из vim сразу после открытия файла?

Правильный ответ: **:**, затем **q**, затем **Enter**

Режим	Назначение
Нормальный	Навигация, удаление, копирование
Вставки	Ввод текста
Командный	Выполнение команд (:q , :w , :%s)

- Для выхода: **:** → **q** → **Enter**

3.1 vim — навигация (w vs W)



Вопрос: Верные утверждения о **w** и **W**

Правильные утверждения: - **W** нельзя переместить курсор на **.** - В строке 9

Клавиша	Тип	Разделители
w	слово (word)	буквы, цифры, _
W	БОЛЬШОЕ СЛОВО (WORD)	только пробелы

- Знаки препинания (**.**, **,**) — разделители для **w**, но не для **W**

3.1 vim — редактирование строки

3.1 Текстовый редактор vim

Шаг 9 из 12

Какие(ой) из предложенных ниже наборов нажатий клавиш выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите один или несколько элементов

x2wwyWpP

xxxxxxxxxwywPp

d2w\$\bifour four <Esc>

d2dwywPp

d2wwifour four <Esc>

d2wwyWpP

Задание: Выбрать наборы клавиш для редактирования

Правильные наборы: - **x2wwyWpP** - **d2w\$\bifour four <Esc>** -

Команда	Действие
d	удалить
y	скопировать (yank)
p	вставить после курсора
P	вставить перед курсором
w	перемещение по словам
\$	в конец строки
i	режим вставки

3.1 vim — замена текста

3.1 Текстовый редактор vim
Шаг 11 из 12

?

?

?

?

?

?

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст (строку)

:%s/Windows/Linux

✓

✓

Всё правильно

↺

Дальше

Решило: 35962

Успешных решений: 61%

Открыть комментарии (74)

Задание: Замена **Windows** → **Linux** (первое вхождение в строке)

Правильный ответ: **:%s/Windows/Linux**

Первое вхождение в строке (по умолчанию)

`:%s/Windows/Linux`

Все вхождения в строке

`:%s/Windows/Linux/g`

С подтверждением

`:%s/Windows/Linux/gc`

- **%** — весь файл

Первое вхождение в строке (по умолчанию)

:%s/Windows/Linux

Все вхождения в строке

:%s/Windows/Linux/g

С подтверждением

:%s/Windows/Linux/gc

- **%** — весь файл
- **s** — substitute

3.2 Bash — история команд

3.2 Скрипты на bash: основы
Шаг 3 из 10

←

?

?

?

?

?

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один элемент

Никакие команды появляться не будут

Из наборов B и C

Из наборов A и C

Только из набора A

Только из набора C ✓

Вопрос: Какие команды будут в истории после вложенных оболочек?

Правильный ответ: Только из набора C

```
bash      # A1, A2, A3
  sh      # B1, B2, B3
    bash  # C1, C2, C3 ← текущая оболочка
```

- Каждая оболочка имеет свою историю

```
bash      # A1, A2, A3
  sh      # B1, B2, B3
    bash  # C1, C2, C3 ← текущая оболочка
```

- Каждая оболочка имеет **свою историю**
- История не наследуется

3.2 Bash — абсолютный путь

3.2 Скрипты на bash: основы

Шаг 5 из 10

?

?

?

?

?

?

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
touch file1.txt
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один элемент

/home/bi/Desktop/file1.txt

/home/bi/Documents/file1.txt

Никак (файла file1.txt не будет существовать после завершения работы скрипта)

/home/bi/file1.txt ✓

Вопрос: Где будет создан **file1.txt**?

Правильный ответ: **/home/bi/file1.txt**

```
#!/bin/bash
cd /home/bi/           # переход в /home/bi
touch file1.txt        # файл создаётся в /home/bi/
cd /home/bi/Desktop/   # переход (не влияет на файл)
```

- Файл остаётся там, где был создан

3.2 Bash — имена переменных

3.2 Скрипты на bash: основы

Шаг 7 из 10

?

?

?

?

?

?

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите один или несколько элементов

VARiable

✓

__variable

✓

var i able

123variable

variable

variable_123

✓

Вопрос: Какие строки могут быть именами переменных?

Правильные варианты: - **VARiable** - **__variable** - **variable** -

?	Можно	?	Нельзя
	Буквы (a-z, A-Z)		Начинаться с цифры
	Цифры (кроме первой позиции)		Спецсимволы (кроме _)
	Знак подчёркивания _		Пробелы

- **123variable** — начинается с цифры → недопустимо

3.3 Bash — циклы и continue

3.3 Скрипты на bash: ветвления и циклы
Шаг 8 из 9

?

▶

?

>.

▶

?

>.

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то сколько раз на экран будет выведено слово "start", а сколько раз слово "finish"?

Выберите один элемент

5 раз "start" и 5 раз "finish"

3 раза "start" и 2 раза "finish"

3 раза "start" и ни разу "finish"

5 раз "start" и 4 раза "finish" ✓

Вопрос: Сколько раз выведется «start» и «finish»?

Правильный ответ: 3 раза «start», 2 раза «finish»

Цикл for — анализ

```
for str in a, b, c_d    # 3 итерации
do
    echo "start"         # всегда (3 раза)
    if [[ $str > "c" ]]
    then
        continue        # пропуск "finish"
    fi
    echo "finish"        # только если условие ложно
done
```

Итерация	\$str	\$str > "c"	«finish»?
----------	--------------	-----------------------	-----------

1	a,	Нет	☐ Да
---	-----------	-----	-----------------------------

2	b,	Нет	☐ Да
---	-----------	-----	-----------------------------

3.4 Bash — вывод echo

3.4 Скрипты на bash: разное
Шаг 5 из 10

▶

?

▶

?

?

▶

?

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash
cd /home/bi/
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один элемент

Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)

/home/bi ✓

pwd

`pwd`

/home/bi/Documents

Вопрос: Что выведет **echo** **"pwd"**?

Правильный ответ: **pwd**

Bash — кавычки и выполнение команд

```
echo "pwd"          # выведет строку "pwd"  
echo $(pwd)         # выведет текущую директорию  
echo `pwd`          # выведет текущую директорию
```

- Двойные кавычки не вызывают выполнение команды

Bash — кавычки и выполнение команд

```
echo "pwd"          # выведет строку "pwd"  
echo $(pwd)         # выведет текущую директорию  
echo `pwd`          # выведет текущую директорию
```

- Двойные кавычки не вызывают выполнение команды
- Для выполнения нужны **`$()`** или обратные кавычки

3.5 Поиск — find -name vs -iname

3.5 Продвинутый поиск и редактирование

Шаг 3 из 13

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star**"`, но **НЕ найдет** команда `find /home/bi -name "star**"`?

Выберите один или несколько элементов

STARS.txt

Eddard_Stark_biography.txt

star_trek OST.mp3

Star_Wars.avi

stardust.mpeg

✓ Всё получилось!

Вопрос: Какие файлы найдёт **-iname**, но не найдёт **-name**?

Правильные ответы: - **STARs.txt** - **Star_Wars.avi**

Опция	Регистр	Пример
-name "star*"	Чувствительный	только star...
-iname "star*"	Нечувствительный	star..., Star..., STAR...

- **-iname** находит все варианты написания

3.5 Поиск — find -path vs -name

3.5 Продвинутый поиск и редактирование
Шаг 4 из 13

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте все верные утверждения из перечисленных ниже.

Выберите один или несколько элементов

- ☐ Опция `-path` используется только для поиска директорий, а `-name` только для поиска файлов
- ☐ Если заменить в команде поиска `-name` на `-path`, то результат поиска всегда останется неизменным
- ☒ Если заменить в команде поиска `-name` на `-path`, то результат поиска иногда может остаться таким же ✓
- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла

✓ Отличное решение!

Дальше

Вопрос: Верные утверждения о **-path** и **-name**

Правильное утверждение: - Если заменить **-name** на **-path**, результат

find — -name vs -path

Опция	Область поиска
-name	Только имя файла
-path	Полный путь к файлу

- Результаты совпадают, когда шаблон находится только в имени файла

3.5 Поиск — find с mindepth/maxdepth

3.5 Продвинутый поиск и редактирование
Шаг 5 из 13

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/
├── dir1
│   ├── file1
│   └── dir2
│       ├── file2
│       └── dir3
│           └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*"`?

Выберите один элемент

☐ Только file1

☒ Все кроме file3 ✓

☐ Все три файла

☐ Ни один файл найден не будет

Вопрос: Какие файлы найдёт **find -mindepth 2 -maxdepth 3**?

Правильный ответ: Ни один файл найден не будет

/home/bi/	(глубина 0)
dir1	(глубина 1)
file1	(глубина 1) ← не подходит
dir2	(глубина 1)
file2	(глубина 1) ← не подходит

- **-mindepth 2** — начинать с глубины 2

find — глубина поиска

/home/bi/	(глубина 0)
dir1	(глубина 1)
file1	(глубина 1) ← не подходит
dir2	(глубина 1)
file2	(глубина 1) ← не подходит

- **-mindepth 2** — начинать с глубины 2
- **file1, file2, file3** на глубине 1 → не найдены

3.5 Поиск — grep -A, -B, -C

3.5 Продвинутый поиск и редактирование
Шаг 7 из 13

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в каждой строке есть слово `"word"`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один элемент

results.txt будет одинакового размера во всех случаях ✓

grep -A 1 "word" file.txt > results.txt и grep -B 1 "word" file.txt > results.txt

grep -C 1 "word" file.txt > results.txt

Все, кроме grep "word" file.txt > results.txt

grep -A 1 "word" file.txt > results.txt

Вопрос: Какая команда создаст наибольший файл?

Правильный ответ: **grep -C 1 "word" file.txt**

Опция	Значение
-A 1	строка + 1 строка после
-B 1	строка + 1 строка до
-C 1	строка + 1 строка до + 1 строка после

- При слове «word» в каждой строке, **-C 1** выводит **все** строки

3.5 Поиск — регулярные выражения grep

3.5 Продвинутый поиск и редактирование
Шаг 9 из 13

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xk1XKL]?[uU]buntu$" text.txt`.

Выберите один или несколько элементов

- Mac OS X 10.9, Windows XP, Ubuntu 12.04
- Well, xubuntu is OK
- Linux is not always Ubuntu ✓
- The best OS is Xubuntu ✓
- Hmm, XKUbuntu ✓
- Lubuntu is better than Ubuntu ✓

✓ Верно. Так держать!

Вопрос: Какие строки выведет **grep -E "[xk1XKL]?[uU]buntu\$"**?

Правильные ответы: - **Well, xubuntu is OK** - **The best OS is Xubuntu**

[xk1XKL]?[uU]buntu\$

Часть	Значение
[xk1XKL]?	0 или 1 символ из набора
[uU]buntu	ubuntu или Ubuntu
\$	конец строки

- Подходит: **Ubuntu, xubuntu, Xubuntu, Lubuntu, Kubuntu**

3.5 Поиск — sed без -n

3.5 Продвинутый поиск и редактирование

Шаг 11 из 13

?

▶

?

▶

?

?

▶

Что произойдет, если в команде `sed -n '/[a-z]*/p' text.txt` не указывать опцию `-n`?

Выберите один элемент

На экран ничего не напечатается

Каждая строчка будет выведена два раза ✓

На экран будет выведено всё содержимое файла text.txt

Появится сообщение об ошибке

✓ Отличное решение!

↺

Дальше

Решило: 29908 Успешных решений: 40%

Вопрос: Что произойдёт без опции **-n**?

Правильный ответ: На экран будет выведено всё содержимое файла

Режим	Поведение
Без -n	Автоматическая печать всех строк
С -n	Печать только по команде p

- Без **-n** все строки выводятся (шаблон **[a-z]*** подходит всем)

3.6 Gnuplot — опция -persist

3.6 Строим графики в gnuplot

Шаг 3 из 10

←

?

?

?

?

?

Вы можете скачать и попробовать применить gnuplot к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске gnuplot, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один элемент

-s, --show-plots-after-exit

Такой опции не существует

-raise

-p, --persist ✓

✓ Всё получилось!

↶

Дальше

Вопрос: Как сохранить графики после закрытия gnuplot?

Правильный ответ: **-p, --persist**

```
# График закроется вместе с gnuplot  
gnuplot script.gp
```

```
# График останется открытым  
gnuplot -p script.gp  
gnuplot --persist script.gp
```

3.6 Gnuplot — autotitle columnhead

3.6 Строим графики в gnuplot

Шаг 5 из 10

?

?

?

?

?

?

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете gnuplot и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один элемент

Название -- первое значение из второго столбца, нарисовано 10 точек

Название "nophone", нарисовано 10 точек

Название "data.csv using 1:2", нарисовано 10 точек

Название -- первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)

Название -- первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена) ✓

Вопрос: Название ряда и количество точек?

Правильный ответ: Название «nophone», 10 точек

```
set key autotitle columnhead  
plot 'data.csv' using 1:2
```

- **columnhead** — использовать первую строку как заголовок

```
set key autotitle columnhead  
plot 'data.csv' using 1:2
```

- **columnhead** — использовать первую строку как заголовок
- Но в **data.csv** нет заголовков (данные с первой строки)

```
set key autotitle columnhead  
plot 'data.csv' using 1:2
```

- **columnhead** — использовать первую строку как заголовок
- Но в **data.csv** нет заголовков (данные с первой строки)
- → Название по умолчанию «none»

3.6 Gnuplot — настройка меток

3.6 Строим графики в gnuplot

Шаг 7 из 10

Предположим, что вы пишете gnuplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде "point <номер точки>, value <значение соответствующей переменной>".

Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20.

Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно!**

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под *конкатенацией* обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст (строку)

set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)

Задание: Установить метки на оси X в точках `x1`, `x2`, `x3`

Правильный ответ:

- "." — конкатенация строк

- "." — конкатенация строк
- Значения переменных подставляются автоматически

3.6 Gnuplot — анимация

3.6 Строим графики в gnuplot

Шаг 10 из 10

▶

?

▶

?

▶

☰

?

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу gnuplot и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в gnuplot работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст (строку)

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Задание: Изменить `move.rot` (отражение, обратное вращение, ускорение)

Модифицированный код:

Требование	Изменение
Зеркальное отражение	Изменение знака приращения угла
Вращение в обратную сторону	+10 вместо +350
Ускорение в 2 раза	pause 0.05 (было 0.1)

3.7 Разное — права доступа (chmod)

3.7 Разное
Шаг 4 из 15

←

?

?

?

?

?

Какая команда(ы) установят файлу `file.txt` права доступа `rw-rw-r--`, если изначально у него были права `r--r--r--`. Укажите **все верные** варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнялись все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите один или несколько элементов

chmod 467 file.txt

chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt ✓

chmod u-wx file.txt; chmod g-w file.txt

chmod rw-rw-r-- file.txt

chmod 777 file.txt

chmod ug+w file.txt; chmod u+x file.txt ✓

Вопрос: Команды для установки прав **rw-rw-r--**

Правильные ответы: - **chmod 467 file.txt** - **chmod a+wx file.txt;**

Целевые права	Владелец	Группа	Остальные
<code>rwXr-xr--</code>	<code>rwX</code> (7)	<code>r-x</code> (5)	<code>r--</code> (4)

- Числовой метод: **chmod 754**? Нет, 467 (признан правильным)

Целевые права	Владелец	Группа	Остальные
rwxr-xr--	rwx (7)	r-x (5)	r-- (4)

- Числовой метод: **chmod 754**? Нет, 467 (признан правильным)
- Буквенный метод: **a+wx**, затем **o-wx**, **g-x**

3.7 Разное — команда wc

3.7 Разное
Шаг 7 из 15

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите один или несколько элементов

- ☒ Количество слов
- ☒ Длину самой длинной строки
- ☒ Количество строк
- ☒ Количество символов
- ☒ Размер файла в байтах

✓ Отлично!

Дальше

Вопрос: Какие характеристики можно посчитать `wc`?

Правильные ответы: все перечисленные

```
wc -l file.txt      # количество строк
wc -w file.txt      # количество слов
wc -c file.txt      # количество байт
wc -m file.txt      # количество символов
wc -L file.txt      # длина самой длинной строки
```

3.7 Разное — размер директории (du)

3.7 Разное
Шаг 8 из 15

?

▶

?

?

▶

?

☰

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо `2048 байт` надо вывести `2.0К`) и **больше** на экран выводить **ничего не нужно**). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по `800 Кбайт` и две поддиректории в каждой из которых лежит по файлу в `400 Кбайт`, то загаданная команда должна вывести на экран одно число: `2.4М` (также на экране может быть выведен еще и символ `,`, обозначающий, что это размер именно *текущей* директории).

Напишите текст (строку)

`du -h -s`

✓

✓

Хорошие новости, верно!

↶

Дальше

Решило: 25177

Успешных решений: 59%

Открыть комментарии (56)

Задание: Вывести размер текущей директории

Правильный ответ: `du -h -s`
2026-05-16 Введение в Linux

```
du -h -s
```

Опция	Значение
-h	human-readable (К, М, G)
-s	summary (только общий размер)

3.7 Разное — создание директорий

3.7 Разное
Шаг 10 из 15

?

?

▶

?

☰

▶

☰

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст (строку)

`mkdir dir{1..3}`

✓

✓ Отлично!

↶

Дальше

Решило: 25501 Успешных решений: 45%

💬

 Открыть комментарии (47)

💡

 Форум решений (19)

Задание: Самая короткая команда для создания `dir1`, `dir2`, `dir3`

Правильный ответ: `mkdir dir{1..3}`

Brace expansion в bash

Длинный вариант

```
mkdir dir1 dir2 dir3
```

Короткий вариант (brace expansion)

```
mkdir dir{1..3}
```

- **{1..3}** раскрывается в **1 2 3**

4. Результаты



- Освоенные темы раздела 3:

- Освоенные темы раздела 3:
 - ▶ Текстовый редактор vim (навигация, редактирование, замена)

- Освоенные темы раздела 3:
 - ▶ Текстовый редактор vim (навигация, редактирование, замена)
 - ▶ Bash-скрипты (переменные, циклы, условия, история)

- **Освоенные темы раздела 3:**
 - ▶ Текстовый редактор vim (навигация, редактирование, замена)
 - ▶ Bash-скрипты (переменные, циклы, условия, история)
 - ▶ Продвинутый поиск (find, grep, sed)

- **Освоенные темы раздела 3:**
 - ▶ Текстовый редактор vim (навигация, редактирование, замена)
 - ▶ Bash-скрипты (переменные, циклы, условия, история)
 - ▶ Продвинутый поиск (find, grep, sed)
 - ▶ Визуализация данных (gnuplot)

- **Освоенные темы раздела 3:**

- ▶ Текстовый редактор vim (навигация, редактирование, замена)
- ▶ Bash-скрипты (переменные, циклы, условия, история)
- ▶ Продвинутый поиск (find, grep, sed)
- ▶ Визуализация данных (gnuplot)
- ▶ Системные утилиты (chmod, wc, du, mkdir)

Навык	Инструменты
Редактирование текста	<code>vim (:w, :q, :%s, w, W, d, y, p)</code>
Bash-скрипты	переменные, for , if , continue
Поиск файлов	find -name, -iname, -path, -mindepth
Поиск текста	grep -E, -A, -B, -C, sed
Визуализация	gnuplot -p, set xtics
Управление файлами	chmod, wc, du -h -s, mkdir {1..3}

5. Заключение

Основные результаты третьего этапа:

- Освоен текстовый редактор vim (навигация, редактирование, замена)

Курс «Введение в Linux» успешно пройден!

Основные результаты третьего этапа:

- Освоен текстовый редактор vim (навигация, редактирование, замена)
- Изучены bash-скрипты (переменные, циклы, условия)

Курс «Введение в Linux» успешно пройден!

Основные результаты третьего этапа:

- Освоен текстовый редактор vim (навигация, редактирование, замена)
- Изучены bash-скрипты (переменные, циклы, условия)
- Приобретены навыки продвинутого поиска (find, grep, sed)

Курс «Введение в Linux» успешно пройден!

Основные результаты третьего этапа:

- Освоен текстовый редактор vim (навигация, редактирование, замена)
- Изучены bash-скрипты (переменные, циклы, условия)
- Приобретены навыки продвинутого поиска (find, grep, sed)
- Освоена визуализация данных в gnuplot

Курс «Введение в Linux» успешно пройден!

Основные результаты третьего этапа:

- Освоен текстовый редактор vim (навигация, редактирование, замена)
- Изучены bash-скрипты (переменные, циклы, условия)
- Приобретены навыки продвинутого поиска (find, grep, sed)
- Освоена визуализация данных в gnuplot
- Изучены системные утилиты (chmod, wc, du)

Курс «Введение в Linux» успешно пройден!